Intelligent Exam Question Analysis

(Milestone 1 – ML-Based Exam Question Analytics)

---

1. Problem Statement

Educational institutions require reliable methods to evaluate the quality and difficulty of exam questions. Traditionally, difficulty estimation is performed manually by educators, which can be subjective, inconsistent, and time-consuming.

The challenge addressed in this project is:

> To design and implement a Machine Learning-based system that automatically predicts the difficulty level of exam questions using textual features and student performance data.

The system must:

- Accept exam question text and response statistics.
- Extract meaningful linguistic and statistical features.
- Train supervised classification models.
- Predict difficulty categories (e.g., Easy, Medium, Hard).
- Provide measurable performance evaluation.

This milestone focuses strictly on classical Machine Learning and NLP techniques without using Large Language Models.

---

2. Data Description

2.1 Dataset Overview

- File Name: `exam_question_dataset_5000.csv`
- Total Records: 5,000 questions
- Task Type: Multi-class classification

The dataset contains structured educational assessment data for predictive analytics.

2.2 Features

1. Textual Feature

- **Question Text**
    - Raw exam question content.
    - Used for linguistic feature extraction.

2. Numerical Features

Derived from student responses:

- Average score
- Score distribution
- Performance variance
- Other response-based statistics

2.3 Target Variable

- **Difficulty Classification**
    - Easy
    - Medium
    - Hard

This is a supervised learning problem where the model predicts the difficulty category.

---

3. Exploratory Data Analysis (EDA) Process

EDA was conducted to understand structure, patterns, and relationships in the dataset before model training.

3.1 Class Distribution Analysis

- Examined frequency of each difficulty label.
- Verified absence of extreme class imbalance.
- Ensured sufficient representation across categories.

Insight: Class distribution was moderately balanced, making the dataset suitable for supervised multi-class classification.

3.2 Textual Analysis

- Calculated average question length.
- Performed word frequency analysis.
- Observed correlation between vocabulary complexity and difficulty.

Key Observations:

- Hard questions often contained domain-specific terminology.
- Easy questions were generally shorter and more direct.
- Medium questions had moderate lexical diversity.

3.3 Numerical Feature Analysis

- Analyzed relationship between average score and difficulty.
- Observed:
    - Lower average scores aligned with harder questions.
    - Higher variance sometimes indicated ambiguous question design.

Insight: Combining textual and statistical features improves predictive strength compared to text-only models.

---

4. Methodology

The system follows a modular architecture:

```
src/
├── preprocessing.py
├── feature_engineering.py
├── train.py
└── evaluate.py
```

4.1 Data Preprocessing

Implemented in `preprocessing.py`.

Steps performed:

- Text lowercasing
- Removal of punctuation
- Stopword removal
- Tokenization
- Label encoding of difficulty categories
- Scaling of numerical features using StandardScaler

Purpose:

- Normalize and clean data before feature extraction.

4.2 Feature Engineering

Implemented in `feature_engineering.py`.

Text Feature Extraction

Used **TF-IDF (Term Frequency – Inverse Document Frequency)**.

Reason for Choosing TF-IDF:

- Efficient for medium-sized datasets.
- Captures the importance of words.
- Works well with linear models.
- Computationally lightweight.

Feature Integration

- TF-IDF matrix (textual representation).
- Scaled numerical response statistics.
- Combined into a single feature matrix for model training.

Artifacts saved:

- `tfidf_vectorizer.pkl`
- `scaler.pkl`
- `label_encoder.pkl`

4.3 Algorithms Used

Implemented in `train.py`.

1. Logistic Regression

- Linear classifier.
- Performs well with sparse high-dimensional TF-IDF features.
- Provides strong baseline performance.

2. Decision Tree

- Captures nonlinear feature interactions.
- Easy to interpret.
- Prone to overfitting without control.

3. Random Forest

- Ensemble of multiple decision trees.
- Reduces overfitting.
- Improves generalization.

Model Selection Strategy:

- All models trained on identical feature sets.
- Performance compared using evaluation metrics.
- Logistic Regression was selected as the final model and stored as `best_model.pkl` based on highest F1-score and accuracy.
- Logistic Regression outperformed ensemble methods, likely due to the high-dimensional sparse nature of TF-IDF features, which favour linear classifiers.

---

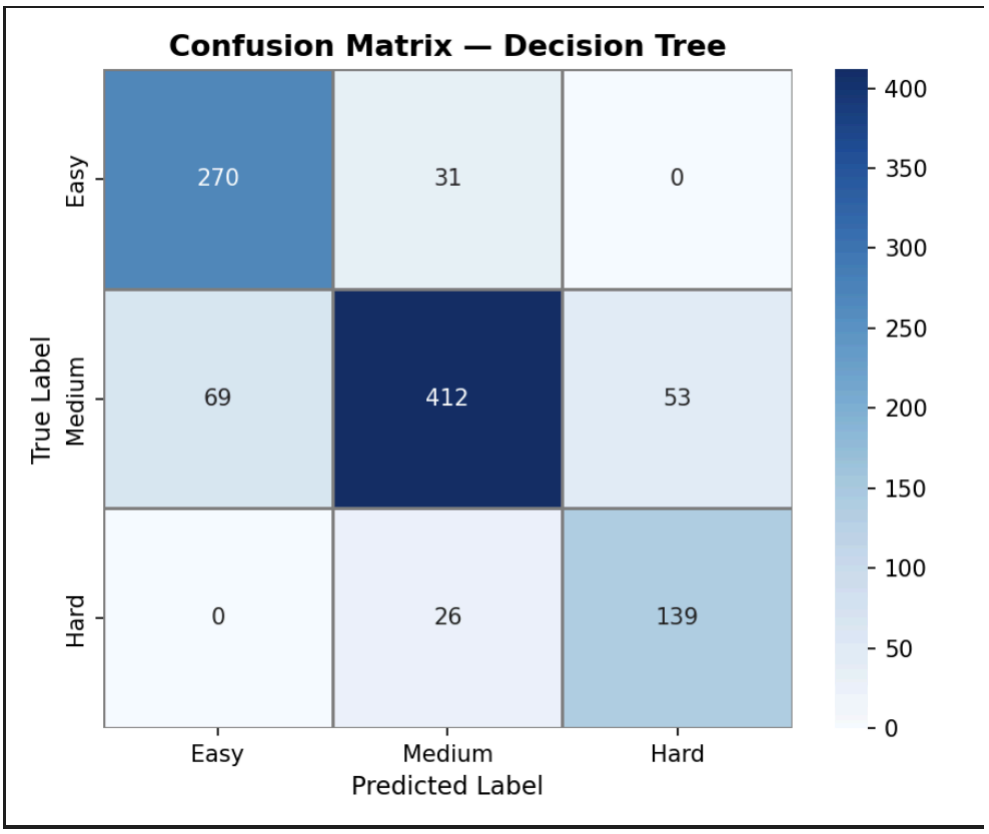
5. Evaluation

Implemented in `evaluate.py`.

5.1 Evaluation Metrics Used

Metric	Purpose
Accuracy	Overall prediction correctness
Precision	Class-specific correctness
Recall	Detection capability per class

F1-Score	Balance between Precision and Recall
Confusion Matrix	Visual class-level performance



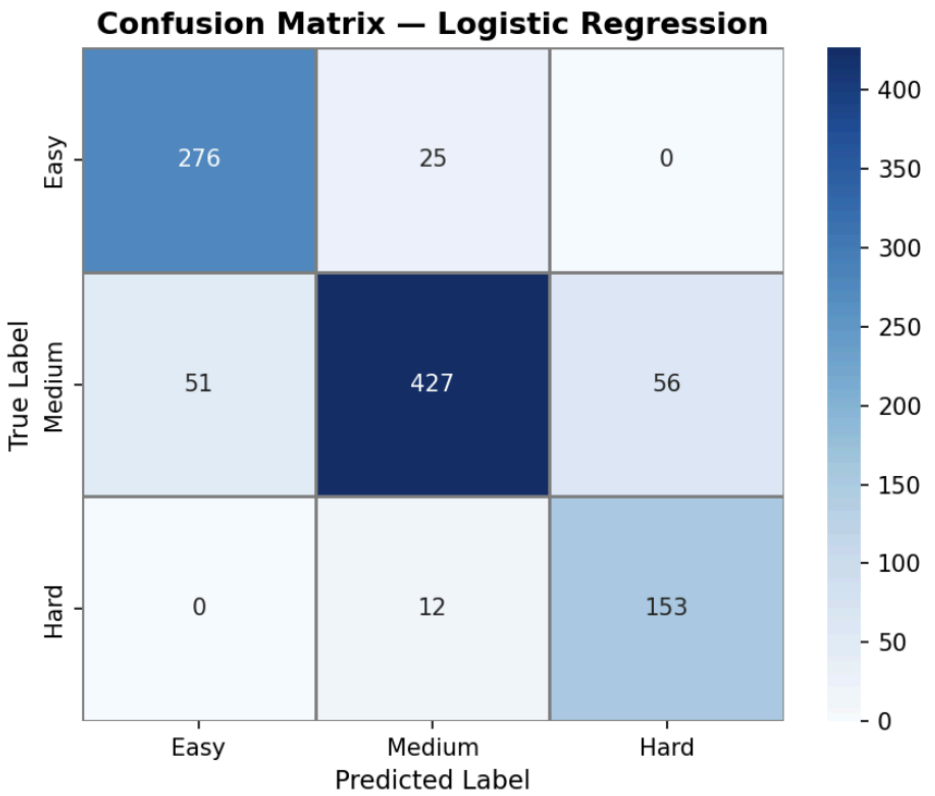
5.2 Results Summary

- Logistic Regression achieved the highest accuracy (85.60%) and F1-score (85.65%), making it the best-performing model.
- Random Forest followed with 83.20% accuracy.
- The Decision Tree showed the lowest performance at 82.10% accuracy with signs of slight overfitting.

The final model was selected based on:

- Highest F1-score
- Stable cross-validation accuracy
- Balanced confusion matrix results

Model	Accuracy	Precision	Recall	F1
Logistic Regression	85.60%	86.63%	85.60%	85.65%
Random Forest	83.20%	85.17%	83.20%	83.28%
Decision Tree	82.10%	82.83%	82.10%	82.12%



All outputs stored in:

- `results_summary.json`
- Confusion matrix visualizations
- `meta.json`

---

6. Optimization

Several techniques were implemented to improve performance.

6.1 Feature Optimization

- Tuned TF-IDF parameters:
  - Maximum features
  - N-gram range
- Removed low-frequency words.
- Scaled numeric inputs for uniformity.

6.2 Hyperparameter Tuning

- Logistic Regression:
  - Logistic Regression: C = 1.0,
  - Random Forest: n_estimators = 100,
  - max_depth = 10
- Decision Tree:
  - Maximum depth
  - Minimum samples split
- Random Forest:
  - Number of estimators
  - Maximum depth

Cross-validation was used for robust performance estimation.

6.3 Overfitting Control

- Controlled tree depth.
- Used ensemble learning (Random Forest).
- Compared training and testing performance.

---

7. User Interface

The web interface was built using:

- Streamlit

Features:

- Input question text.
- Load trained model.
- Display predicted difficulty.
- Show evaluation metrics.
- Interactive and modular design.

This ensures usability and readiness for deployment.

Figure 1:

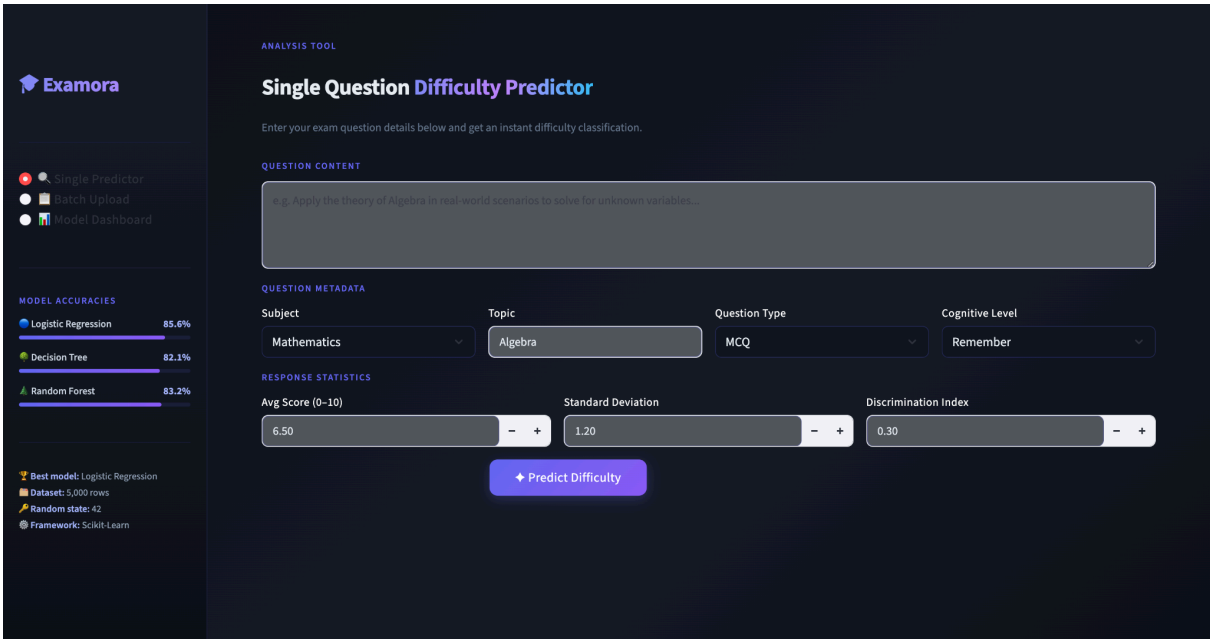
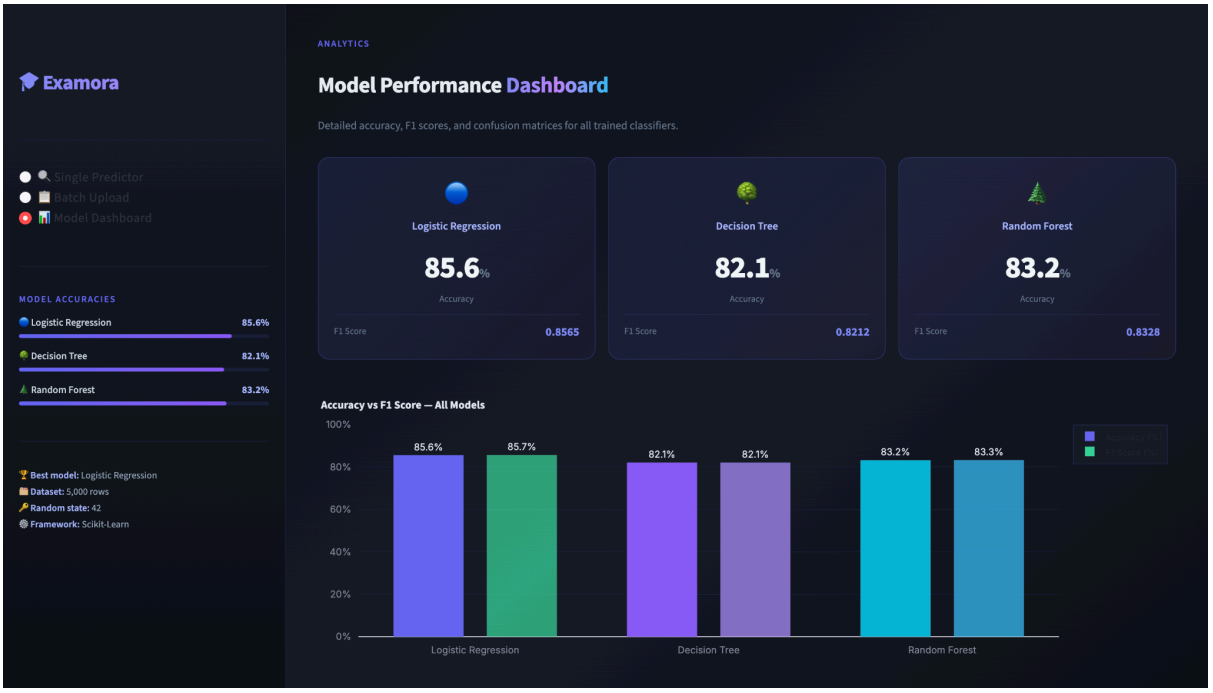


Figure 2:



8. Team Contribution

Team Member	Contribution
Yuvraj Chandravansi	Responsible for model creation, including preprocessing, feature engineering (TF-IDF, scaling), training, and evaluation.
Siddhant Giri	Assisted with data preprocessing and cleaning, and prepared the project report.
Joshit Dutta	Assisted with dataset preparation and helped in UI development using Streamlit.

Work distribution ensured equal technical involvement and modular development.

---

9. Conclusion

Milestone 1 successfully demonstrates:

- Proper application of NLP techniques for feature extraction.
- Integration of textual and numerical data.
- Comparison of multiple classical ML models.
- Performance evaluation using standard classification metrics.
- Optimization and model selection strategy.

The final Logistic Regression model achieved 85.60% accuracy and 85.65% F1-score, demonstrating strong predictive capability for exam question difficulty classification.